FACE LANDMARK AND SEGMENTATION ANALYZER

CSC 126 – Graphics and Visual Computing | Short Project Documentation

Section:

BJY21

Submitted by:

Exaure, Joseph A.

Idea, Irene Rose T.

Latosa, Jhon Loyd P.

Submitted to:

Joshua Tubo

October 13, 2026

1. Project Description

This project is a Python and OpenCV application, built in Google Colab, that takes one static face image and shows the detected face, its facial landmark mesh, and the segmented skin, face and lip regions.

2. Project Objectives

- Prepare the image with image operations: resizing, grayscale conversion, histogram equalization and blurring.
- Detect the face and its facial landmarks as a dense mesh of points.
- Segment the skin, face and lip regions, using the landmarks as a guide.
- Show every stage of the result with masks, bounding boxes, labels and overlays.

3. Problem / Real-World Use-Case

Schools and offices often ask people to upload a single photo, for example for a student ID, an enrollment form or an online application, and staff then must check by eye that each photo shows a clear face. This is slow and inconsistent, and some photos show no face, several faces, or a face that is turned or partly hidden. A static image fits this problem because the photo is taken once and uploaded, so there is no video to follow, and one analysis of that single image is enough to judge it without tracking the face over time. The project finds how many faces a photo contains, marks the facial landmarks (eyes, eyebrows, nose, mouth and face outline), and separates the face skin and lips from the background. These results are the base for deciding whether a photo is usable; the project only shows the results and does not accept or reject photos.

4. Techniques Used

The Colab notebook runs these steps in order: load and preprocess the image, detect the face landmarks, segment, then visualize.

Image Operations

- cv2.resize scales the image to at most 800 px wide, keeping its aspect ratio.
- cv2.cvtColor converts the image between color spaces: BGR to RGB for the landmark model, to grayscale, and to YCrCb for segmentation.
- cv2.equalizeHist spreads out the gray levels to improve contrast, and cv2.GaussianBlur (5x5) smooths the color image before the skin threshold.
- cv2.bitwise_and and cv2.addWeighted combine masks with the image to produce the results and overlays.

Image Segmentation

- Skin: cv2.cvtColor converts the image to YCrCb, and cv2.inRange keeps pixels inside a skin-color range (Y 0-255, Cr 133-173, Cb 77-127).

- Cleanup: cv2.morphologyEx with an elliptical 5x5 kernel applies opening to remove specks, then closing (2 iterations) to fill small holes.
- Face region: the filled outline of the face-oval landmarks (including the forehead) is used as a mask. The final face-skin mask is the skin mask combined with this region, so the landmarks guide the segmentation.
- Lips: the filled outline of the lip landmarks is used as the lip mask.
- cv2.findContours and cv2.drawContours outline the segmented face-skin region.

Facial Landmark Detection

- MediaPipe Face Landmarker detects each face and returns a dense mesh of 478 landmark points per face, in normalized coordinates.
- The coordinates are converted to pixel positions, and the face bounding box is taken from the smallest and largest landmark positions.
- The landmarks are drawn as green points, with a red bounding box and a "Face 1" label.

5. Screenshots of Results

This section presents screenshots of the application during testing. Each screenshot should be taken from the actual running application.

5.1 Images Use



Figure 1. Raw photo of the group member used to test.

5.2 Sample 1

```
Downloading face landmark model (one time only)...
Model saved to models/face_landmarker.task
Faces detected: 1
  Face 1: 478 landmark points
Saved summary figure: outputs/sample1_summary.png
```

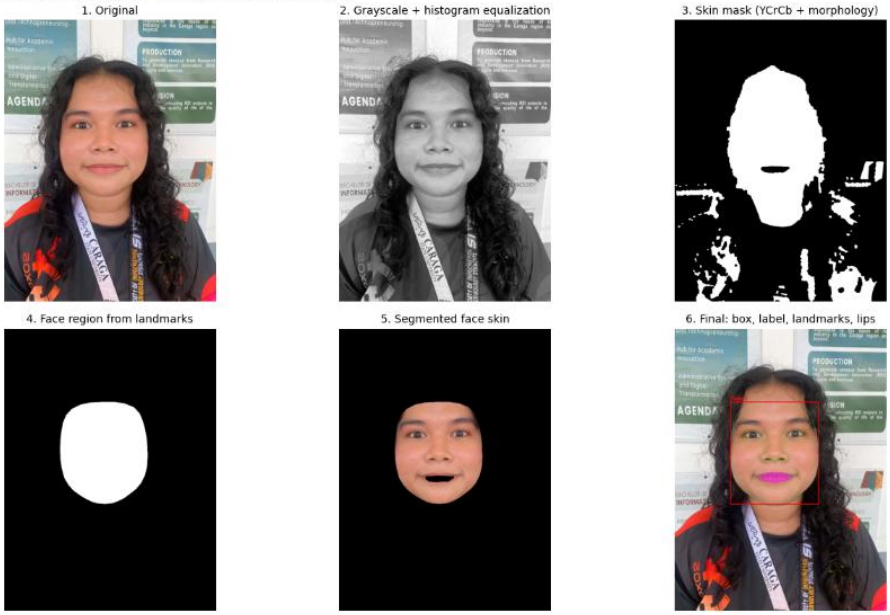


Figure 2. Sample 1 results for (Image Operation, Image Segmentation, Facial Landmark).

5.3 Sample 2

```
Faces detected: 2
  Face 1: 478 landmark points
  Face 2: 478 landmark points
Saved summary figure: outputs/sample2_summary.png
```

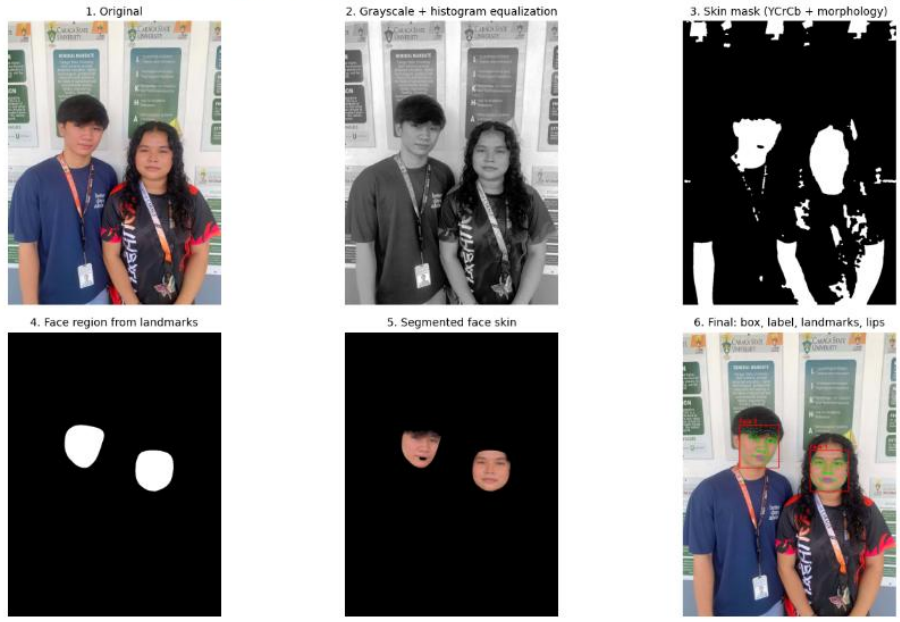


Figure 3. Sample 2 results for (Image Operation, Image Segmentation, Facial Landmark).

5.4 Sample 4

```
Faces detected: 3
  Face 1: 478 landmark points
  Face 2: 478 landmark points
  Face 3: 478 landmark points
Saved summary figure: outputs/sample3_summary.png
```

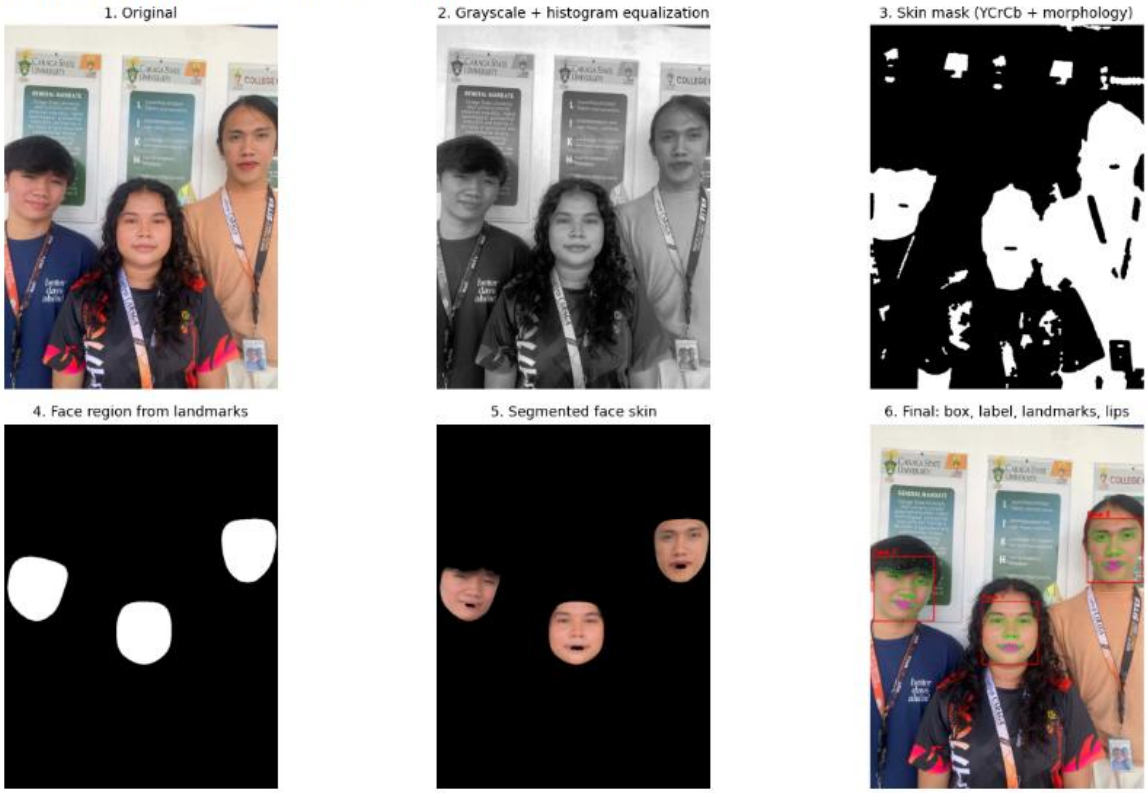


Figure 4. Sample 3 results for (Image Operation, Image Segmentation, Facial Landmark).

6. Observed Behavior and Accuracy Discussion

Accuracy and behavior. We did not measure accuracy against labeled data, so the results are visual checks on our three sample photos. Every person's face was detected and each face received 478 landmark points that sat on the eyes, eyebrows, nose, mouth and face outline, with the lip overlay on the lips. The notebook reads one image at a time, prints the number of faces found and the landmark count for each, shows the six-panel figure, and saves the results to the outputs folder; if no face is detected, it prints a message instead. The skin mask alone was less clean, because it also picked up skin-colored areas outside the faces, such as the tan shirt in the third photo, but combining it with the face region from the landmarks kept only the faces in the final result.

7. Limitations

- The skin threshold uses one fixed YCrCb range, so skin-colored clothing, strong shadows or unusual lighting can add false regions or leave gaps in the skin mask. The final result depends on the landmark face region to keep only the face.
- Hair, the lips and the mouth are not skin-colored, so they appear as dark gaps in the segmented face.

- The detector can miss small faces: in a photo with seven people, it found only two. The program detects up to 10 faces per image, and we did not evaluate tilted, turned or partly covered faces.
- The lip mask is the filled outline of the lip landmarks, so it also covers the mouth opening when the mouth is open.
- The notebook processes one static image at a time. It runs in Google Colab, which needs internet, and Colab deletes the uploaded photos and results when the session ends unless they are downloaded.

8. Conclusion

The project combines Image Operations, Image Segmentation and Facial Landmark Detection in one Python pipeline built with OpenCV and MediaPipe that works on a static image and shows each stage of its result. On our three sample photos it detected every person's face (1 of 1, 2 of 2 and 3 of 3), with 478 landmark points on each face. Combining the skin mask with the landmark face region kept the final segmentation on the faces only, even though the skin mask alone also picked up skin-colored areas such as a tan shirt.

GitHub Repository:

https://github.com/irenerosie-web/csc-126-midterm-project?fbclid=IwY2xjawU7GH1leHRuA2FlbQIxMABwZG9mBWJyaWQRMWNCbWU4WEljR2lmbE5oQlFzcnRjBmFwcF9pZBAyMjIwMzkxNzg4MjAwODkyAAEeF_MDGgfPG4aETAwHc8cQ2FIglzY1EFrXoRzSovoNU-igWf0TAzvqO2YzsBY_aem_wnMyeIjIyu-5JSJAKMt_mA